1. Self-Evolving Computer Architecture

Development of a Self-Evolving Computer Architecture for Autonomous Runtime Optimization and Reconfiguration of Computing Resources

Design an architecture that can **observe workload behavior during execution and dynamically modify its computing resources or architectural configuration** to improve performance or resource utilization. The proposed design must define how runtime information is collected, how decisions are made, and how the architecture is reconfigured without treating the system as a completely new architecture for every workload.

The proposed architecture must be **simulated as an interacting system**, showing the runtime monitoring, decision-making, reconfiguration mechanism, and computing resources, along with the overhead introduced by adaptation.

Context example: An architecture executing a workload whose behavior changes from compute-intensive to memory-intensive during execution should be able to recognize the change and adapt its configuration accordingly.

---

2. Distributed Computer Architecture

Development of a Distributed Computer Architecture for Unified Management of Physically Separated Compute, Memory, Storage, and Accelerator Resources

Design an architecture in which **compute, memory, storage, and accelerator resources are physically separated but can be dynamically organized and managed as a unified computing system**. The design must address how resources are discovered, allocated, communicated with, and reconfigured at runtime while accounting for the latency and bandwidth costs of accessing remote resources.

The proposed architecture must be **simulated with the individual resources, interconnect, resource-management mechanism, and workloads interacting with each other**, demonstrating whether dynamic resource composition is feasible and what overhead it introduces.

Context example: A system running several applications simultaneously may dynamically allocate additional memory or accelerator resources to one application while reallocating unused resources from another.

3. Runtime-Composable Chiplet Architecture

Development of an Intelligent Runtime-Composable Chiplet Architecture for Workload-Specific Computing Systems

Design a chiplet-based architecture in which **different compute, memory, cache, or accelerator chiplets can be dynamically composed or reconfigured at runtime according to the requirements of the executing workload**. The design must specify how chiplets communicate, how composition decisions are made, and how the system handles the latency, bandwidth, and reconfiguration overhead associated with changing the composition.

The proposed architecture must be **simulated as a complete interacting system**, demonstrating the chiplets, interconnect, runtime composition mechanism, workload execution, and the transition between different configurations.

Context example: A system may require one configuration while processing a large matrix workload and a different configuration when subsequently executing a graph-processing workload.

---

4. Memory-Centric Computing Architecture

Development of a Memory-Centric Computing Architecture for Data-Local Execution and Intelligent Computation Placement

Design an architecture that can **dynamically decide where computation should take place based on the location, movement, reuse, and characteristics of data**. The architecture should reduce unnecessary data movement by allowing computation to be placed closer to the data when beneficial, while retaining the ability to use conventional compute resources when that is more efficient.

The proposed architecture must be **simulated with memory, compute resources, data movement, computation placement, and runtime decision-making explicitly interacting**, allowing the feasibility and overhead of dynamic computation placement to be evaluated.

Context example: The same system may process a highly reusable dataset, a streaming dataset, and a randomly accessed dataset, requiring different computation-placement decisions during execution.

5. Energy-Aware Adaptive Computer Architecture

Development of an Energy-Aware Adaptive Computer Architecture for Dynamic Optimization of Computation, Memory, and Data Movement Under Variable Power Constraints

Design an architecture that can **dynamically change its computation, memory, and data-movement behavior in response to changing power or energy constraints** while maintaining an acceptable level of performance. The design must determine how the architecture monitors its power state, makes runtime decisions, and transitions between different operating configurations.

The proposed architecture must be **simulated with the power-management mechanism, computing resources, memory system, data movement, runtime controller, and workload interacting**, demonstrating whether the architecture can remain within changing power constraints and the performance cost of doing so.

Context example: A workload may execute under different available power budgets during its execution, requiring the architecture to continuously adapt its resource usage rather than operating with a fixed configuration.